4-bit Array Multiplier from RTL to GDSII on SkyWater SKY130 with Open-Source Tools

Sudhanva Joshi (2023102022) Aditya Peketi (2024122001)
Digital VLSI Design — RTL-to-GDSII Assignment

***Abstract*—We design a 4-bit × 4-bit unsigned array multiplier in Verilog with registered inputs and output, verify it exhaustively, and take it through synthesis (Yosys), static timing analysis (OpenSTA) and place and route (LibreLane) on the SKY130 `sky130_fd_sc_hd` library. The RTL and the gate-level netlist both pass all 256 test vectors. Synthesis produces 61 cells ($757.0\,\mu\text{m}^2$, 52.9 % of it flip-flops); STA at the typical corner gives $T_{min} = 2.79$ ns ($F_{max} = 358.4$ MHz). The final layout, at a 3.5 ns clock, has 0 DRC errors, 0 LVS errors and $+0.077$ ns setup slack, for a post-route F_{max} of 292.2 MHz.**

I. Design

A. Array structure

For every bit b_i of B there is a partial product $pp_i[j] = a[j] \cdot b[i]$, shifted left by i. Row 0 consists only of AND gates: $p[0] = pp_0[0]$, and $r_0 = \{0, pp_0[3{:}1]\}$ is passed down. Rows 1–3 are 4-bit ripple-carry adders. Full adder j of row i adds $pp_i[j]$, $r_{i-1}[j]$ and the carry $c_i[j]$ (with $c_i[0] = 0$). The lowest sum bit of each row is a finished product bit ($p[i] = s_i[0]$), and $r_i = \{c_i[4], s_i[3{:}1]\}$ is passed to the next row. Row 3 also gives $p[7{:}4] = \{c_3[4], s_3[3{:}1]\}$. The array therefore uses $N^2 = 16$ AND gates and $N(N-1) = 12$ full adders (Fig. 1).

B. RTL

`full_adder` is written with two `assign` statements: $sum = a \oplus b \oplus c_{in}$ and $c_{out} = ab + ac_{in} + bc_{in}$. `mult_array` registers a and b into `a_r` and `b_r` on every rising edge, builds the three adder rows with `generate` loops (12 `full_adder` instances, no $\star$ operator) and registers the 8-bit result into p. All 16 flip-flops have an asynchronous active-low reset. An input applied before edge 1 is captured at edge 1 and the product appears in p at edge 2, so the latency is 2 cycles and the throughput is one result per cycle.

II. Part A — RTL Design and Verification

A. Testbench

`tb_mult` generates a 10 ns clock, holds `rst_n` low for the first two cycles and then applies all 256 combinations of a and b. Inputs change on the falling edge; after two rising edges the testbench compares p with $a \times b$ ($\star$ is used only here) half a cycle later, using `!==` so that an unknown output also counts as an error. It prints a line per mismatch, a single summary line, and writes `mult.vcd`. The RTL result is:

```
PASS: 256 vectors checked, 0 errors
```

B. Error-detection check

With `sum` in `full_adder.v` temporarily changed to `a ^ b` (carry-in ignored), the same testbench reports 95 mismatches, the first being a=3, b=3, p=5 (expected 9):

```
FAIL: 256 vectors checked, 95 errors
```

The change was then undone and the testbench passes again.

C. Hand-worked example

We used Sudhanva Joshi's roll number (2023102022): its last digit is 2, so $A = 2 + 5 = 7$, and since $A = 7$ the assignment gives $B = 9$.

$$
\begin{array}{rll}
 & 0\,1\,1\,1 & A = 7 \\
\times & 1\,0\,0\,1 & B = 9 \\
\hline
 & 0\,1\,1\,1 & pp_0\ (b_0 = 1) \\
 & 0\,0\,0\,0 & pp_1\ (b_1 = 0),\ \text{shift } 1 \\
 & 0\,0\,0\,0 & pp_2\ (b_2 = 0),\ \text{shift } 2 \\
 & 0\,1\,1\,1 & pp_3\ (b_3 = 1),\ \text{shift } 3 \\
\hline
 & 0\,0\,1\,1\,1\,1\,1\,1 & P = 63
\end{array}
$$

Through the array: $r_0 = 0011$; row 1 adds $pp_1 = 0000$, giving $s_1 = 0011$, $p[1] = 1$, $r_1 = 0001$; row 2 adds $pp_2 = 0000$, giving $s_2 = 0001$, $p[2] = 1$, $r_2 = 0000$; row 3 adds $pp_3 = 0111$, giving $s_3 = 0111$, $c_3[4] = 0$, $p[3] = 1$ and $p[7{:}4] = 0011$. Hence $P = 0011\,1111_2 = 63$.

D. Waveform

Fig. 2 shows the example in the RTL simulation. b changes from 8 to 9 at 2440 ns (a falling edge, $a = 7$ throughout). Rising edge 1 at 2445 ns (marker A) captures a and b into the input registers; rising edge 2 at 2455 ns (marker B) captures the product, and p changes from 56 to 63. The latency is two clock cycles, 20 ns.

III. Part B — Logic Synthesis

`synth/synth.tcl` reads both RTL files, runs `synth -top mult_array -flatten`, maps flip-flops with `dfflibmap` and logic with `abc` to the typical-corner Liberty file, removes unused wires, and writes `mult_array_stat.rpt` and the netlist (Table I). The flip-flop cell `sky130_fd_sc_hd__dfrtp_1` has an area of $25.024\,\mu\text{m}^2$.

The gate-level netlist was simulated with the SKY130 cell models (`-DFUNCTIONAL -DUNIT_DELAY=#1`) and the same testbench; it passes all 256 vectors, so synthesis preserved the behaviour. The netlist also contains two `lpflow_isobufsrc_1` cells, which `abc` chose freely because the script does not exclude any library cells.

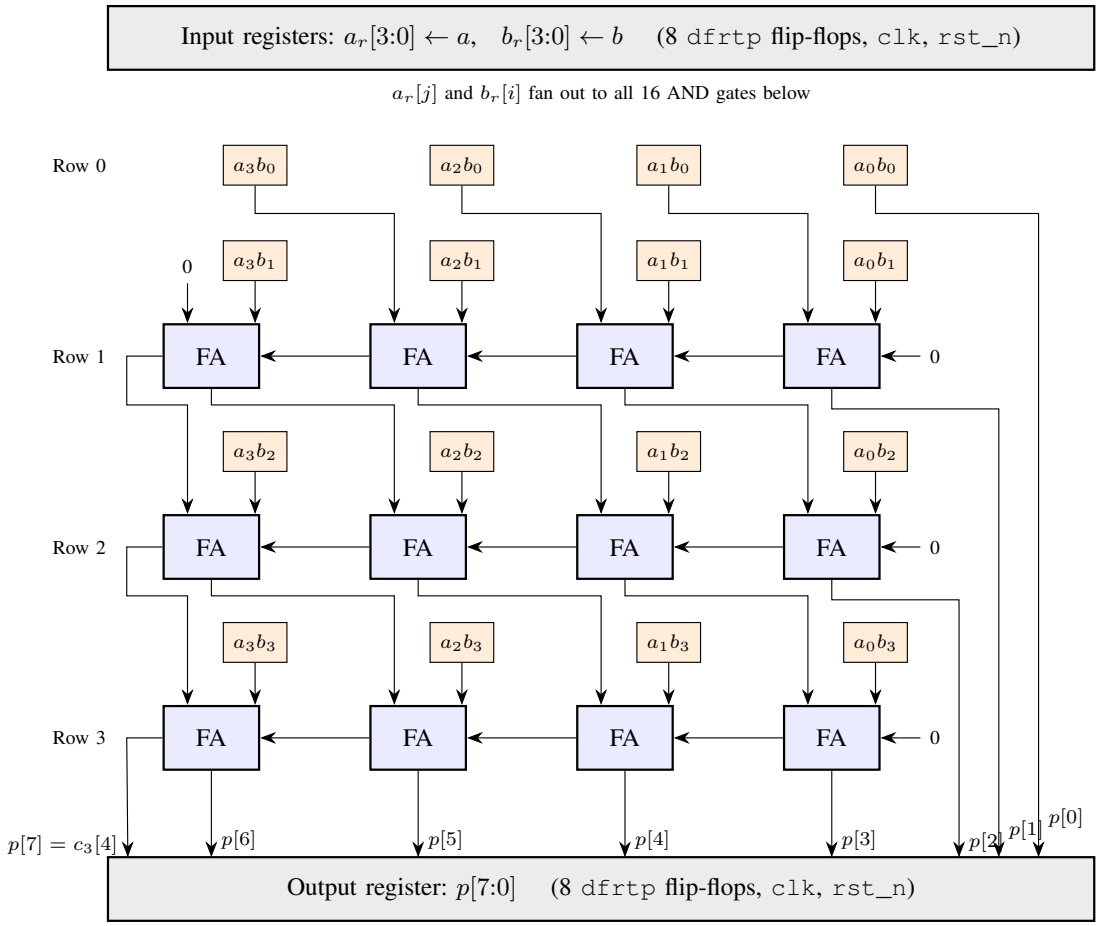


Fig. 1. Block diagram of `mult_array`: 16 AND gates ($a_jb_i = pp_i[j]$), 12 full adders in three ripple-carry rows, and the input and output registers (16 flip-flops). Each full adder takes $pp_i[j]$ (right input) and $r_{i-1}[j]$ (left input); carries ripple from right to left.

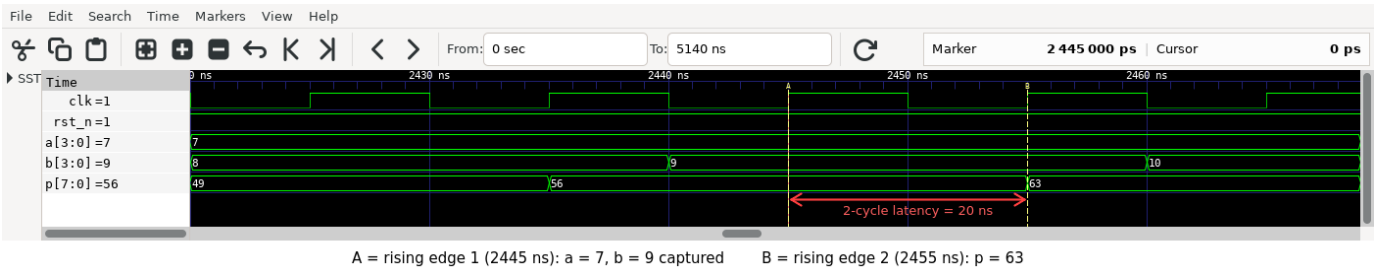


Fig. 2. GTKWave view of `clk`, `rst_n`, a, b and p (decimal) for 7×9. Markers A and B are the two rising edges of the 2-cycle latency.

IV. Part C — Static Timing Analysis

`sta/sta.tcl` reads the typical-corner library (`tt_025C_1v80`) and the netlist, creates the clock, sets 1.0 ns input and output delays, and reports the worst setup slack and the critical path. At $T = 10$ ns the worst slack is 7.213 ns, so $T_{min} = 10 - 7.213 = 2.787 \approx 2.79$ ns and $F_{max} = 1000/2.79 = 358.4$ MHz. Rerunning at $T = 2.79$ ns gives a slack of $+0.003$ ns (Table II).

The critical path (Table III) runs from the register holding $a_r[1]$ to the register capturing $p[6]$. Five of its nine logic cells are recognisable full-adder cells: three `maj3` (carry) and `xnor3`/`xor3` (sum). The first three cells implement the partial products and the simplified first adder row, which `abc` merged into complex gates.

V. Part D — Physical Design

A. Configuration and runs

`pnr/config.json` sets `DESIGN_NAME`, the two RTL files, `CLOCK_PORT`, `FP_SIZING = "absolute"`, a 150×150 µm die, and `CLOCK_PERIOD` $= 1.2 \times 2.79 = 3.348 \rightarrow 3.5$ ns.

TABLE I
PART B RESULTS

Metric	Value
Total number of cells	61
Number of flip-flops	16
Total cell area	756.976 µm^2
Flip-flop area (16×25.024)	400.384 µm^2
Flip-flop area (% of total)	52.89 %
3 most-used cell types:	
`dfrtp_1`	16
`nand2_1`	7
`maj3_1`, `and2_0` (tied)	4 each
Netlist simulation:	
`PASS: 256 vectors checked, 0 errors`	

TABLE II
PART C RESULTS

Metric	Value
Worst slack at $T = 10$ ns	7.213 ns
T_{min}	2.79 ns
F_{max}	358.4 MHz
Slack at $T = T_{min}$	+0.003 ns
Critical path start point	`_91_` ($a_r[1]$ flip-flop)
Critical path end point	`_88_` ($p[6]$ flip-flop)
Cells on the critical path	9 logic (+2 flip-flops)
Latency $= 2 \times T_{min}$	5.58 ns
Throughput	358.4 M results/s

TABLE III
CRITICAL PATH AT $T = 2.79$ NS

#	Cell	Role	Delay (ns)
–	`dfrtp_1` (`_91_`)	launch, CLK→Q	0.398
1	`clkinv_1`	partial product logic	0.073
2	`o311ai_0`	partial product / row 1	0.356
3	`a21oi_1`	row 1 logic	0.153
4	`maj3_1`	FA carry	0.402
5	`xnor3_1`	FA sum	0.370
6	`maj3_1`	FA carry	0.368
7	`maj3_1`	FA carry	0.354
8	`xor3_1`	FA sum	0.178
–	`dfrtp_1` (`_88_`)	capture, setup	0.135
	Data arrival time		2.652

TABLE IV
LIBRELANE RUNS (SLACK IN NS)

Run	Period	Corners	Worst	Typical	Result
1	3.5	all 9	−2.184	0.511	fail
2	4.0	all 9	−1.684	1.011	fail
3	4.5	all 9	−1.184	1.511	fail
4	5.0	all 9	−0.682	2.011	fail
5	5.5	all 9	−0.208	2.509	fail
6	6.0	all 9	−0.154	2.877	fail
7	6.5	all 9	−0.224	3.117	fail
8	7.0	all 9	−0.137	3.518	fail
9	3.5	typical	0.077	0.077	**pass**

All runs: 0 DRC errors, 0 LVS errors.

LibreLane signs off by default at nine process corners. Run 1 met timing at the typical corner (+0.51 ns) but failed at the slow corner (ss, 100 °C, 1.60 V) by −2.18 ns. Following the assignment, we increased the period in 0.5 ns steps (Table IV). From 5.5 ns onwards the slow-corner slack stopped improving and stayed between −0.14 and −0.22 ns. The post-CTS timing repair only optimises for the typical corner, so with a longer period it does less resizing and the slow-corner path slows down as the clock relaxes. Since Part C specifies the typical corner (`tt_025C_1v80`) only, the final run restricts signoff to that corner with `"STA_CORNERS": ["nom_tt_025C_1v80"]` at 3.5 ns. This run passes.

B. Results

Table V lists the final metrics from `pnr/metrics.json`. The post-route maximum frequency is $F_{max} = 1000/(3.5 - 0.077) = 1000/3.423 = 292.2$ MHz, 18 % lower than the 358.4 MHz from Part C. The post-route critical path runs from the $a_r[0]$ register to the $p[6]$ register through eight logic cells, four of them full-adder cells (`xor3`, `maj3` ×2 and `xnor3`).

C. Cells added by the flow

The 1882 instances break down as in Table VI. LibreLane re-synthesises the RTL itself and produces 62 logic and flip-flop cells (one more than Part B, because it does not use the `lpflow` cells); everything else was added during physical design.

D. Layout

Fig. 3 shows the final GDS in KLayout. In the whole-die view the 150 µm die, the standard-cell rows and the I/O pins on the die edges are visible; the signal routing is concentrated in the middle of the core, where the 78 logic, flip-flop and buffer cells sit, and the rest of the rows hold decap and fill cells. The zoomed view shows the rows bounded by horizontal power rails, the poly gates inside the cells, horizontal metal 1 and vertical metal 2 wires, and the vias between them.

VI. ANSWERS TO THE QUESTIONS

Q1. Synthesis produced $61 - 16 = 45$ logic cells for the array, instead of $16 + 12 = 28$ blocks. The count is not a one-to-one mapping because `synth -flatten` dissolves the full-adder boundaries and `abc` re-optimises the whole cone of logic. Some full adders are split into a sum cell (`xor3`, `xnor3`, `xor2`) plus a carry cell (`maj3`) or several smaller gates, which raises the count. Others shrink: adders with a constant-0 input ($c_i[0] = 0$, $r_0[3] = 0$) reduce to half adders, and AND gates are merged into complex cells such as `a21oi`, `a22oi` and `o311ai`. The library's full-adder cell `sky130_fd_sc_hd__fa_1` has two outputs, which `abc` cannot map to (it reports it as a multi-output cell and skips it), so each full adder costs at least two cells.

Q2. The critical path at $T_{min} = 2.79$ ns runs from the input-register flip-flop holding $a_r[1]$ to the output-register flip-flop capturing $p[6]$. It contains nine logic cells between the

TABLE V
PART D RESULTS (FINAL RUN)

Metric	Value
`CLOCK_PERIOD` used	3.5 ns
Number of runs needed	9
Core area (die area)	17 759.5 µm^2 (22 500)
Cells after synthesis (Part B)	61
Cells after place and route	1882
Kinds of cells added	decap, fill, tap, buffers
Wirelength	2495 µm
Setup slack	+0.077 ns
Post-route F_{max}	292.2 MHz
F_{max} from Part C	358.4 MHz
DRC errors (Magic / KLayout)	0 / 0
LVS errors	0

TABLE VI
CELLS AFTER PLACE AND ROUTE

Cell kind	Count
Design logic and flip-flops	62
Decap (`decap_3` … `decap_12`)	1300
Fill (`fill_1`, `fill_2`)	253
Well taps (`tapvpwrvgnd_1`)	245
Timing-repair buffers (`buf_2`, `clkdlybuf4s50_1`)	19
Clock-tree buffers (`clkbuf_16`)	3
Total	1882

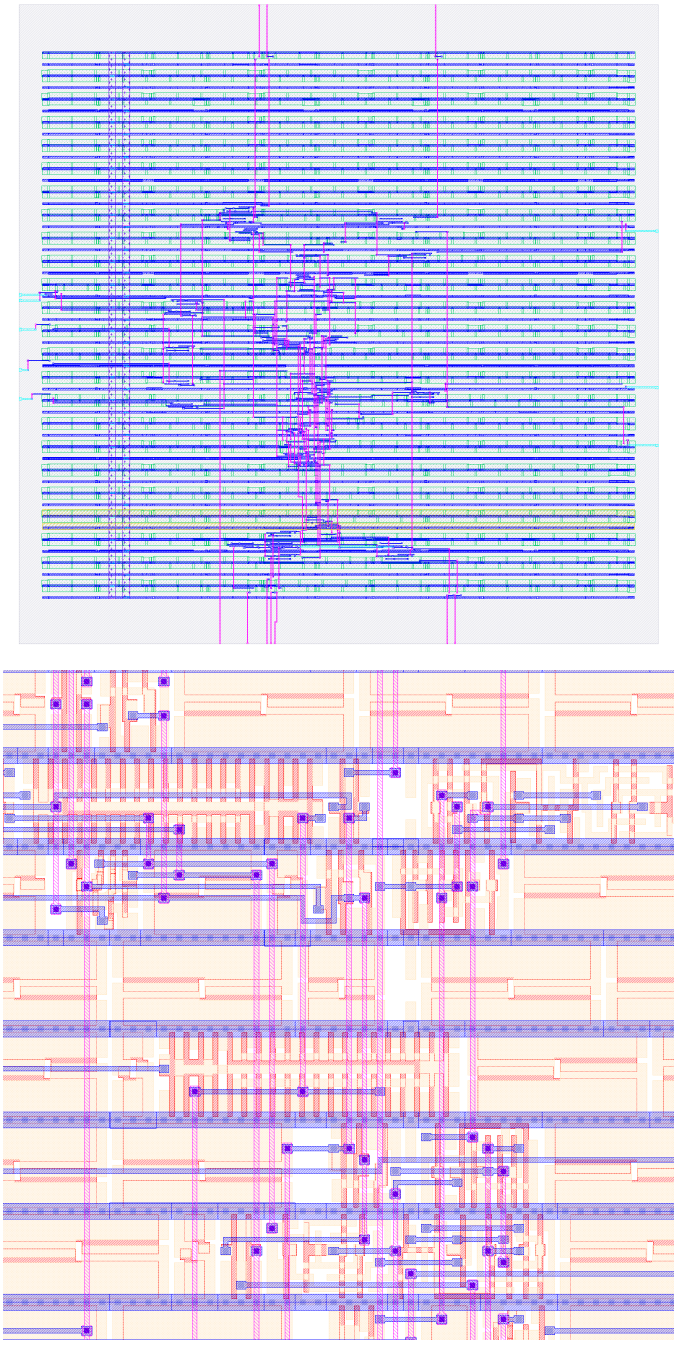


Fig. 3. Final layout in KLayout. Top: whole die (150 × 150 µm; metal layers, n-well and boundary). Bottom: zoomed view of about 20 × 20 µm (poly, local interconnect, metal 1, metal 2 and vias).

two flip-flops (Table III), of which five are full-adder cells. The estimate of $2N = 8$ full-adder delays describes the unoptimised array. Here the first stages are simplified (half adders and merged AND logic in three fast complex gates), so only about five full-adder-like stages remain. These five cells take 1.672 ns (about 0.33 ns each) and the three first-stage gates 0.582 ns, so the 2.254 ns of logic delay corresponds to roughly seven full-adder delays: close to, and slightly below, the $2N = 8$ estimate.

Q3. Flip-flops are 52.89 % of the total cell area (400.384 of 756.976 µm^2). A `dfrtp_1` with asynchronous reset is 25.0 µm^2, about seven times a `nand2_1` (3.75 µm^2), because it contains two latches, a clock inverter and reset logic. The design needs 16 of them (8 input bits and 8 output bits), while the combinational array is only 45 small gates. The register count grows linearly with N but the array grows with N^2, so the flip-flop share is largest for small multipliers.

Q4. Post-route F_{max} is 292.2 MHz versus 358.4 MHz in Part C. Several effects reduce it:

- The routed wires add resistance and capacitance, extracted after routing.
- The flow adds a 0.25 ns clock uncertainty and a 5 % timing derate.
- The inputs are driven by a real cell and the outputs drive a real load.

The launch-to-capture data path grows from 2.652 ns to 3.03 ns. The flow added five kinds of cells (Table VI):

- Clock-tree buffers (`clkbuf_16`) deliver the clock to all 16 flip-flops with low skew. They also add 0.32 ns of insertion delay, which is common to the launch and capture flip-flops.
- Timing-repair buffers fix slew and capacitance limits and timing.
- Tap cells tie the n-wells and substrate to VPWR/VGND to prevent latch-up.

- Decap cells add on-chip capacitance between the supply rails, reducing supply noise when cells switch.
- Fill cells close the remaining gaps so that the wells, implants and power rails are continuous, as DRC requires.

Q5. A DRC (design rule check) verifies that the layout geometry obeys the foundry's manufacturing rules: minimum widths, spacings, enclosures, overlaps and densities on every layer. An LVS (layout versus schematic) check extracts the transistors and their connections from the layout and compares them with the netlist. This confirms that the layout implements exactly the intended circuit, with no shorts, opens or missing devices. Both must pass before fabrication:

- A DRC violation can make the chip impossible to manufacture reliably, for example by causing shorts or broken wires and low yield.
- An LVS mismatch means a perfectly manufacturable chip that does the wrong thing.

Our final layout has 0 DRC errors (Magic and KLayout) and 0 LVS errors (Netgen).

Q6. For $N = 8$:

- **Size.** The array needs $N^2 = 64$ AND gates and $N(N-1) = 56$ full adders, against 16 and 12 for $N = 4$: $56/12 \approx 4.7$ times as many full adders. The logic area therefore grows roughly with N^2.
- **Path length.** The critical path is about $2N = 16$ full-adder delays instead of 8, twice as long; delay grows linearly with N.
- **Delay estimate.** From Part C, the 4-bit logic delay is $2.652 - 0.398 = 2.254$ ns. Scaling it by $16/8$ gives about 4.5 ns of logic for the 8-bit path. Adding the clock-to-Q (0.40 ns) and setup (0.14 ns) times gives $T_{min} \approx 5.0$ ns, or $F_{max} \approx 200$ MHz (versus 358 MHz).
- **Registers.** The design would need $8 + 8 + 16 = 32$ flip-flops.

VII. Conclusion

The 4-bit array multiplier was taken from RTL to a DRC- and LVS-clean SKY130 layout. It is functionally correct in RTL and gate-level simulation, runs at 358 MHz before layout and 292 MHz after routing at the typical corner, and occupies a 150×150 µm die. All commands, in order, are listed in the repository README.